

MSc 2025-2026 · Data Architecture for AI

Labo Santé

Architecture de données containerisée
pour un projet IA

Livrable Session 1 — Modélisation & premiers conteneurs
Aline Maire · 9 juin 2026

Sommaire

1. Cahier des charges
2. Schéma SQL (PostgreSQL) — export dbdiagram.io
3. Schéma NoSQL (MongoDB) — document JSON
4. Diagramme des services & interactions

1. Cahier des charges

1.1 Contexte

Un **laboratoire de santé fictif** souhaite centraliser ses données médicales aujourd'hui éclatées entre plusieurs fichiers et logiciels métier. L'objectif est de bâtir une plateforme :

- **fiable** — intégrité référentielle, pas de perte au redémarrage
- **scalable** — capable d'ingérer plusieurs millions de consultations
- **administrable** — interfaces web pour explorer les données
- **observable** — monitoring temps réel des services
- **portable** — tout est containerisé, déployable en une commande

À terme, ces données alimenteront des modèles IA (prédiction de prescription, détection d'interactions médicamenteuses, NLP sur les consultations).

1.2 Données à gérer

Domaine	Type	Stockage cible	Justification
Médecins	structuré	PostgreSQL	schéma stable, jointures
Patients	structuré	PostgreSQL	schéma stable, intégrité forte
Médicaments	structuré	PostgreSQL	référentiel, peu volatile
Prescriptions	structuré (N-N)	PostgreSQL	relations patient ↔ médecin ↔ médicament
Consultations	semi-structuré	MongoDB	symptômes/diagnostics de longueur et structure variables, pas de jointures nécessaires

1.3 Besoins fonctionnels

- **F1** — Enregistrer/consulter médecins, patients, médicaments via une UI (pgAdmin).
- **F2** — Créer des prescriptions liant patient + médecin + 1..N médicaments.
- **F3** — Stocker les consultations (symptômes libres, diagnostic structuré, notes) avec lien `patient_id` vers PostgreSQL.
- **F4** — Ingérer en masse des données simulées (jusqu'à 6 M de consultations) pour tester la volumétrie.
- **F5** — Re-exécuter le pipeline sans créer de doublons (idempotence).
- **F6** — Consulter en temps réel l'état des services et leurs logs (Portainer).

1.4 Besoins techniques

Catégorie	Exigence
Conteneurisation	<code>docker compose up -d</code> démarre tous les services
Persistance	volumes Docker pour Postgres et Mongo (survie à <code>down</code>)
Réseau	un réseau Docker isolé, communication inter-services par nom de service (<code>postgres</code> , <code>mongo</code> ...)
Secrets	aucun mot de passe en dur — variables via fichier <code>.env</code> (jamais commité)
Ordonnancement	<code>depends_on</code> + <code>healthcheck</code> pour démarrer l'ingestor seulement quand les bases sont prêtes
Observabilité	pgAdmin (Postgres UI), Mongo Express (Mongo UI), Portainer (Docker UI), <code>docker stats</code>
Reproductibilité	<code>.env.example</code> , <code>requirements.txt</code> , <code>Dockerfile</code> versionnés ; <code>.gitignore</code> strict

1.5 Périmètre

Inclus : modélisation, infra Docker, pipeline d'ingestion Python, monitoring infra, documentation.

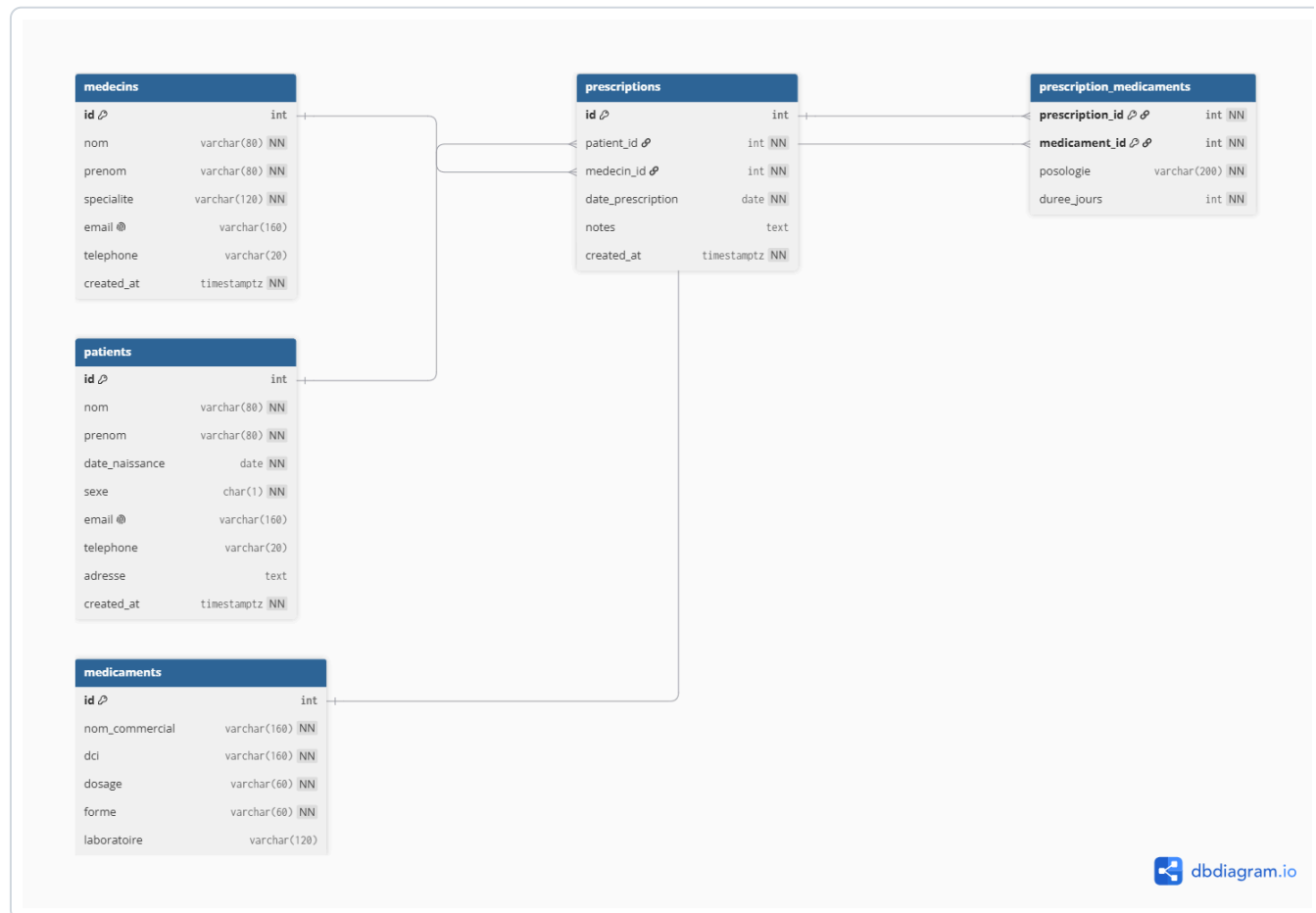
Hors périmètre : authentification utilisateurs finaux, frontal métier, RGPD opérationnel (données 100 % synthétiques pour le TP), entraînement de modèles IA.

1.6 Critères d'acceptation

- `docker compose up -d` démarre tous les services sans erreur.
- pgAdmin accessible sur `localhost:5050` , Mongo Express sur `localhost:8081` , Portainer sur `localhost:9000` .
- Schéma SQL créé automatiquement au premier démarrage (`init.sql`).
- Ingestion de 60 000 / 600 000 / 6 000 000 lignes mesurée (temps + RAM).
- Aucune donnée perdue après `docker compose down && docker compose up -d` .
- Aucun secret en clair dans le dépôt Git.

2. Schéma SQL (PostgreSQL)

Source dbdiagram.io éditable : <https://dbdiagram.io/d/Labo-Sante-Schema-SQL-6a27e6f88eb8ca4bfe863a2b>



2.1 Tables

Table	Rôle	Clés
medecins	référentiel praticiens	PK <code>id</code> , UNIQUE <code>email</code>
patients	référentiel patients	PK <code>id</code> , UNIQUE <code>email</code> , CHECK <code>sexe</code> IN ('M', 'F', 'X')
médicaments	référentiel médicaments	PK <code>id</code> , UNIQUE (<code>nom_commercial</code> , <code>dosage</code> , <code>forme</code>)
prescriptions	actes de prescription	PK <code>id</code> , FK <code>patient_id</code> , FK <code>medecin_id</code>
prescription_medicaments	liaison N-N	PK composite (<code>prescription_id</code> , <code>medicament_id</code>)

2.2 Contraintes & index

- **FK** avec `ON DELETE RESTRICT` (sauf cascade sur la liaison N-N pour permettre la suppression d'une prescription).
- **CHECK**: `duree_jours > 0`, `sexe IN ('M','F','X')`.
- **Timestamps**: `created_at TIMESTAMPTZ DEFAULT NOW()`.
- **Index**: `patient_id`, `medecin_id`, `date_prescription`, `dci`.
- **Auto-exécution**: `schema.sql` est monté dans `/docker-entrypoint-initdb.d/` → DDL appliquée au tout premier `docker compose up`.

3. Schéma NoSQL (MongoDB)

Collection unique `consultations` . Exemple de document :

```
{
  "_id": "ObjectId('64f...')",
  "patient_id": 1428,
  "medecin_id": 12,
  "date": "2026-06-09T09:30:00Z",
  "motif": "Toux persistante depuis 10 jours",
  "symptoms": [
    { "label": "toux sèche",          "duree_jours": 10, "intensite": "moderee" },
    { "label": "fièvre",             "duree_jours": 3,  "temperature_c": 38.4 },
    { "label": "douleur thoracique", "intensite": "legere" }
  ],
  "diagnosis": {
    "primary": { "code_cim10": "J20.9", "label": "Bronchite aiguë" },
    "secondary": [ { "code_cim10": "R05", "label": "Toux" } ],
    "severity": "moderee",
    "confidence": 0.82
  },
  "antecedents_pertinents": ["asthme léger 2019"],
  "notes": "Auscultation : râles bronchiques bilatéraux. Pas de signe de gravité.",
  "examens_demandes": ["NFS", "CRP"],
  "attachments": [
    { "type": "audio", "filename": "auscult.wav", "url": "s3://lab/2026/06/09/abc.wav" }
  ],
  "created_at": "2026-06-09T09:42:11Z"
}
```

3.1 Champs obligatoires

Champ	Type	Rôle
<code>patient_id</code>	int	lien vers <code>patients.id</code> (PostgreSQL) — jointure applicative
<code>medecin_id</code>	int	lien vers <code>medecins.id</code> (PostgreSQL)
<code>date</code>	ISODate	date de la consultation
<code>symptoms</code>	array<object>	longueur variable
<code>diagnosis</code>	object	structure variable

3.2 Index recommandés

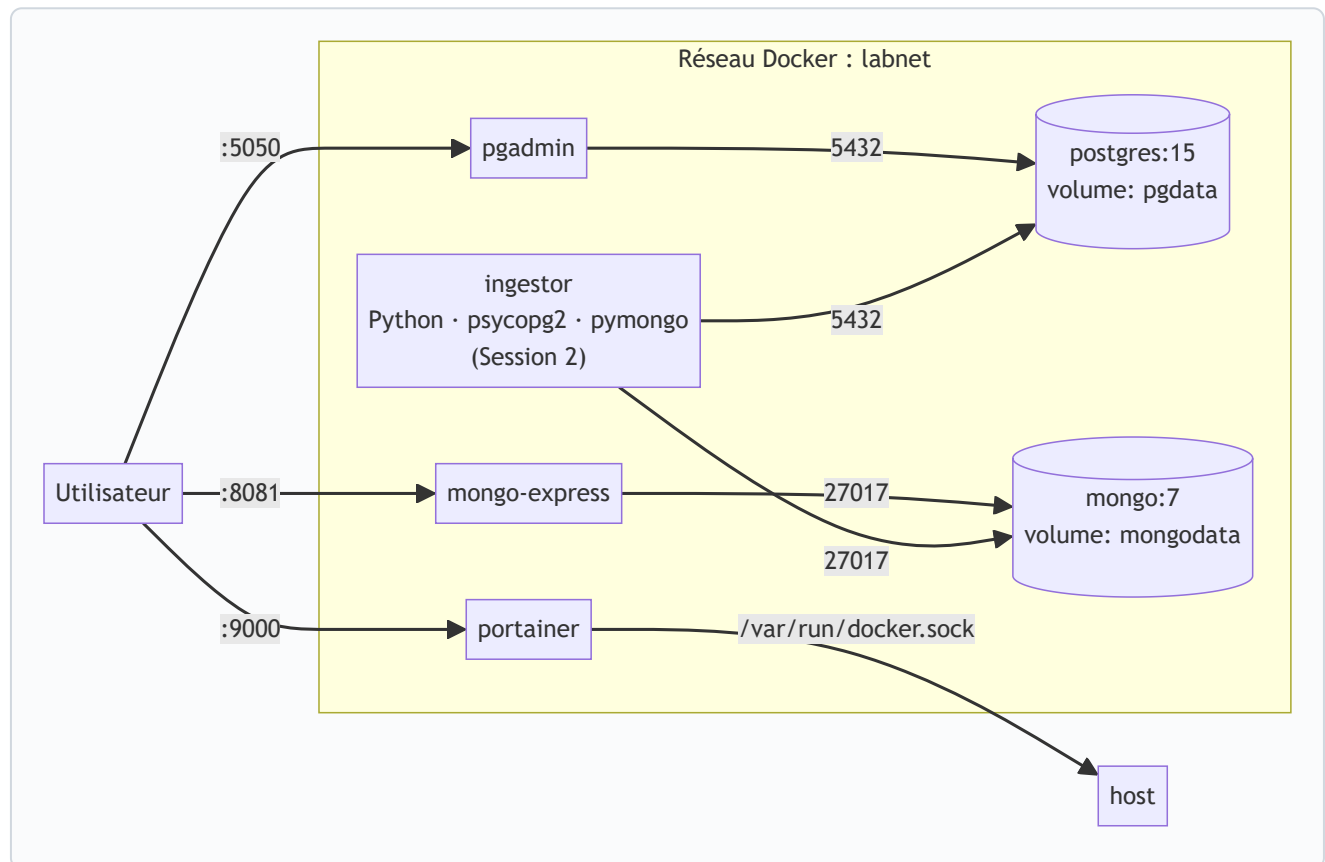
- `{ patient_id: 1 }` — toutes les consultations d'un patient
- `{ date: -1 }` — chronologie inverse
- `{ "diagnosis.primary.code_cim10": 1 }` — recherche par diagnostic CIM-10

3.3 Justification NoSQL

- `symptoms` et `diagnosis` ont une **longueur et une structure variables** (impossible à modéliser proprement en SQL sans 5+ tables).
- **Pas de jointure complexe** nécessaire sur ce document — toute lecture se fait par `patient_id` ou `date`.
- `patient_id` assure le **lien logique** avec PostgreSQL (jointure applicative côté ingestor / API).
- Évolution facile du modèle : ajouter `examens_demandes`, `attachments` ... **sans migration**.

4. Diagramme des services & interactions

Tous les services tournent dans un **réseau Docker isolé** (`labnet`). Ils communiquent par leur **nom de service** (pas `localhost`). Les volumes nommés (`pgdata` , `mongodata` , `pgadmin` , `portainer`) assurent la persistance entre `docker compose down / up` .



4.1 Inventaire des services (Session 1)

Service	Image	Port hôte	Volume	Dépend de
postgres	postgres:15-alpine	127.0.0.1:5432	pgdata	—
mongo	mongo:7	127.0.0.1:27017	mongodata	—
pgadmin	dpage/pgadmin4	127.0.0.1:5050	pgadmin	postgres (healthy)
mongo-express	mongo-express:1	127.0.0.1:8081	—	mongo (healthy)
portainer	portainer/portainer-ce	127.0.0.1:9000	portainer	—

(Le service `ingestor` arrive en Session 2.)

4.2 Healthchecks

- **postgres** : `pg_isready -U $POSTGRES_USER -d $POSTGRES_DB`
- **mongo** : `mongosh --eval "db.adminCommand('ping')"`

Les UIs (`pgadmin` , `mongo-express`) ne démarrent qu'après `condition: service_healthy` sur leur base — évite les erreurs « connection refused » au boot.

4.3 Sécurité

- Aucun mot de passe en dur : tout passe par `.env` (non commité).
- `.env.example` documente les variables sans valeurs sensibles.
- Les ports admin sont exposés **uniquement sur** `127.0.0.1` (durcissement de base, à compléter en prod).